

Beginner Guide to Running SaltStack Across Multiple Servers

Managing individual servers manually becomes impossible as your infrastructure grows. SaltStack helps you automate and execute commands across hundreds of servers simultaneously in seconds.

What you'll learn:

- 1 Checking Server Connectivity Across Your Fleet
- 2 Targeting Specific Servers Using System Grains
- 3 Running Ad-Hoc Shell Commands Globally
- 4 Enforcing System Configuration with States

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Checking Server Connectivity Across Your Fleet

SaltStack works using a Master-Minion structure where one central server controls many worker servers. Before making changes, you always test if your worker servers are online and communicating. The test.ping module sends a fast heartbeat check to all targeted nodes and returns a true or false response.

```
$salt '*' test.ping
```

Cloud & DevOps Daily

How SaltStack Runs Commands Across 100 Servers at Once

Sunday, September 13, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Targeting Specific Servers Using System Grains

Grains are system properties automatically discovered on target servers, such as OS type, CPU architecture, or domain name. Instead of running commands everywhere, you can target specific subsets of infrastructure. This prevents accidental execution of commands on the wrong server types.

```
$salt -G 'os:Ubuntu' cmd.run 'apt-get update'
```

Cloud & DevOps Daily

How SaltStack Runs Commands Across 100 Servers at Once

Sunday, September 13, 2026

Swipe to tip 3 >> [linkedin.com/in/mrmanrajsingh](https://www.linkedin.com/in/mrmanrajsingh)

3

TIP 3 OF 4

Running Ad-Hoc Shell Commands Globally

When troubleshooting incidents, you often need instant operational data from your servers, such as current memory usage or system uptime. The `cmd.run` execution module lets you run standard Linux commands across all targeted minions and streams the output directly back to your terminal.

```
$salt '*' cmd.run 'df -h'
```

Cloud & DevOps Daily

How SaltStack Runs Commands Across 100 Servers at Once

Sunday, September 13, 2026

Swipe to tip 4 >> [linkedin.com/in/mrmanrajsingh](https://www.linkedin.com/in/mrmanrajsingh)

4

TIP 4 OF 4

Enforcing System Configuration with States

Running one-off commands works for quick fixes, but real infrastructure requires consistent configuration states. Salt state files (.sls) define how a system should look, such as requiring Nginx to be installed and running. Salt checks the server, finds differences, and automatically fixes them.

```
$salt '*' state.apply webserver
```

Cloud & DevOps Daily

How SaltStack Runs Commands Across 100 Servers at Once

Sunday, September 13, 2026

See Key Takeaways on next page >> linkedin.com/in/mrmanrajsingh

Key Takeaways

- + Always verify target matching using `test.ping` before running execution commands.
- + Check pending minion authentication keys regularly using `salt-key -L`.
- + Use Salt state files instead of ad-hoc shell commands for permanent changes.
- + Group servers logically using custom Grains or Roles rather than IP addresses.
- + Keep Salt Master and Minion package versions aligned across your fleet.

Watch Out For

- ! Using the wildcard '*' target in production without double-checking your scope.
- ! Forgetting to accept newly added server keys on the Salt Master using `salt-key -a`.
- ! Writing non-repeatable shell scripts inside `cmd.run` instead of declarative states.
- ! Blocking required SaltStack communication ports (4505 and 4506) on firewalls.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh